

SOC Insider Threat Detection

Using Wazuh and Machine Learning

Technical Project Report

File Integrity Monitoring and Anomaly Detection in a Simulated SOC Environment

Nakedi Tumelo Peta

Cybersecurity Intern | 4IR Lab

Contents

1. Project Overview.....	3
2. Problem Statement.....	3
3. Project Objectives	3
4. Lab Architecture.....	3
4.1 Architectural Representation.....	4
Why Docker.....	5
Agents in the lab	5
5. File Integrity Monitoring with Syscheck.....	5
6. Simulated Insider Behaviour	5
Why large files point to an insider	5
7. Machine Learning Approach	6
Model used: Isolation Forest	6
Features used for training.....	6
8. Wazuh Visualisations	7
9. Business Use Cases.....	8
10. Key Takeaways	8
11. Future Improvements	9
12. Conclusion	10
13. References	11

1. Project Overview

This project builds a small Security Operations Centre (SOC) lab that detects insider threats through file activity. Insiders already have legitimate access, so their actions rarely trip the signature based rules that catch outside attackers. The lab takes a different route. It watches what files users touch, learns what normal looks like, and flags the behaviour that does not fit.

The setup combines two layers. Wazuh handles file integrity monitoring and centralised log collection across several simulated users. An Isolation Forest machine learning model then reviews the file events and isolates the ones that look abnormal, without anyone writing a rule for each possible attack.

2. Problem Statement

Most security tooling is tuned to spot intruders coming from outside the network. Insider threats break that assumption. The person is supposed to be there. They have a valid account, valid credentials, and a reason to access company systems. When that same person starts copying large volumes of data or moving files they normally never touch, traditional rule based detection often stays quiet because no signature has been violated.

The gap this project addresses is simple to state and hard to close: how do you catch a trusted user behaving in an untrusted way. The answer here is to stop looking for known bad signatures and start looking for deviations from each user's own normal pattern.

3. Project Objectives

The lab was built to meet a few concrete goals:

- Deploy a working SOC environment that monitors file activity across multiple users in real time.
- Simulate realistic insider behaviour, such as a user suddenly creating unusually large files in a sensitive directory.
- Apply machine learning to detect that behaviour as an anomaly, rather than relying on fixed rules.
- Visualise the results in Wazuh so an analyst can triage alerts quickly.

4. Lab Architecture

The environment runs entirely in Docker. Each container plays a role you would find in a real SOC, which keeps the lab light enough to run on a single machine while still mirroring how the pieces fit together.

Component	Role in the lab
Docker containers	Simulated end users generating file activity
Wazuh Manager	Central SOC brain that collects and analyses events
Syscheck	File Integrity Monitoring engine inside Wazuh

Component	Role in the lab
Isolation Forest	Machine learning model that flags anomalies

4.1 Architectural Representation



Why Docker

Docker lets each simulated user run in its own isolated container while sharing one host. Spinning users up and tearing them down takes seconds, the environment resets cleanly between tests, and the whole lab stays reproducible. That matters when you want to run the same insider scenario several times and compare results.

Agents in the lab

Four Wazuh agents were registered, three acting as users and one as the manager:

- user_day_1 – a normal user with routine file activity
- user_day_2 – a second normal user, used to establish a baseline
- insider – the agent that performs the suspicious behaviour
- wazuh-server – the manager that receives and processes all events

5. File Integrity Monitoring with Syscheck

Syscheck is the Wazuh module that watches files and directories for change. It is the sensor layer of this whole project. Every time a file is created, modified, or deleted inside a monitored path, Syscheck records the event and ships it to the manager.

In this lab Syscheck monitors sensitive paths such as /data, where company information would realistically live. The events it captures fall into three types:

- File creation, when a new file appears in a watched directory.
- File modification, when an existing file changes.
- File deletion, when a file is removed.

File creation turned out to be the most useful signal for this scenario. An insider preparing to steal data usually has to gather it first, which means writing new files, often large ones, into a directory. Watching for that pattern gives early warning before the data ever leaves the building.

6. Simulated Insider Behaviour

To test detection, the insider agent was made to behave the way a real data thief might. Instead of touching a few small files like a normal user, it created large files inside the monitored /data path. This stands in for someone aggregating documents, database dumps, or media before exfiltration.

Why large files point to an insider

Large file writes are unusual for ordinary day to day work. When a user who normally edits small documents suddenly produces a file many times bigger than their baseline, that size jump is worth a closer look. In security terms it can signal:

- Data aggregation, where information is being collected into one place.

- Intellectual property theft, where proprietary files are being copied.
- Preparation for data leakage, where a payload is being staged before it leaves the network.

None of these is proof of wrongdoing on its own. The point of the system is to surface the behaviour so a human analyst can investigate, not to convict automatically.

7. Machine Learning Approach

Rule based detection needs someone to define every bad pattern in advance. That does not scale to insider threats, because the same action can be normal for one user and alarming for another. Machine learning sidesteps the problem by learning each environment's normal behaviour and flagging whatever deviates from it. No predefined signature is required.

Model used: Isolation Forest

The lab uses an Isolation Forest, an unsupervised algorithm built specifically to find outliers. It works by randomly partitioning the data and measuring how easily each point can be separated from the rest. Anomalies get isolated in very few splits, which is exactly what makes them stand out. The model suits security data well because abnormal events are rare and do not come neatly labelled, which is the situation a real SOC faces every day.

Features used for training

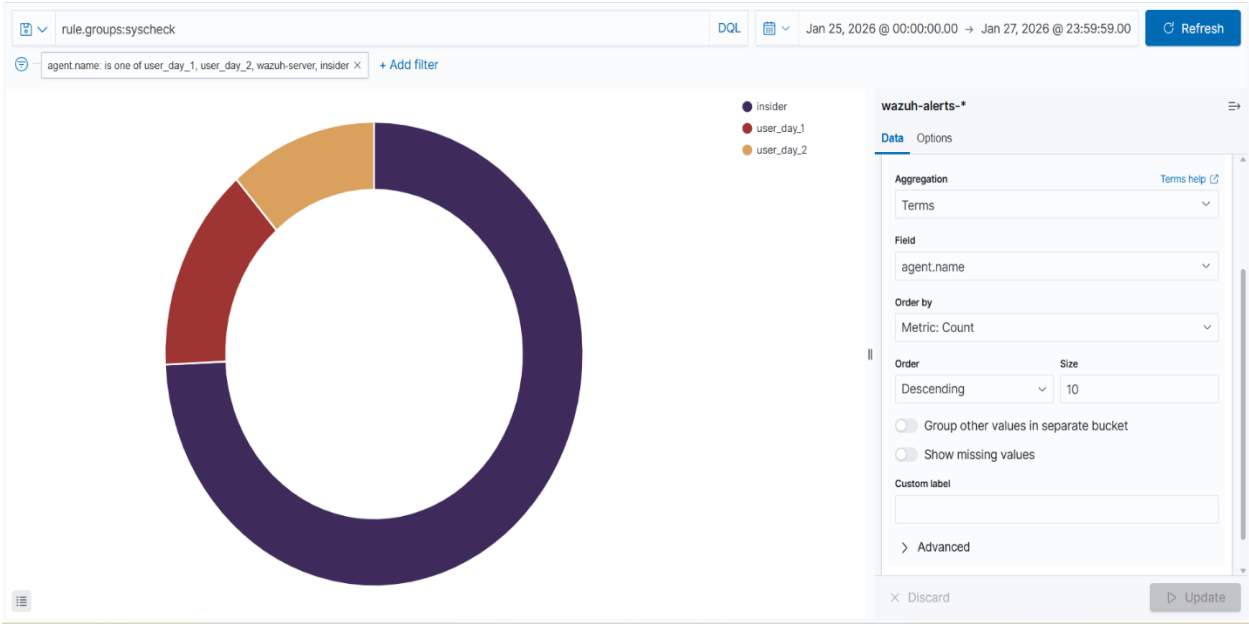
Each file event was turned into a small set of numerical features the model could learn from:

Feature	What it captures
File size	Volume of data written, the strongest insider signal
Event type	Whether the file was created or modified
File path depth	How deep in the directory tree the activity sits
Privilege level	Whether the action ran as root or a normal user
Permission length	The permission string on the affected file

8. Wazuh Visualisations

Detection is only useful if an analyst can read it fast. The Wazuh dashboard presents the results in three views that together let you go from a high level count down to a single suspicious event:

- Pie charts breaking down alerts per agent, so one noisy user stands out immediately.

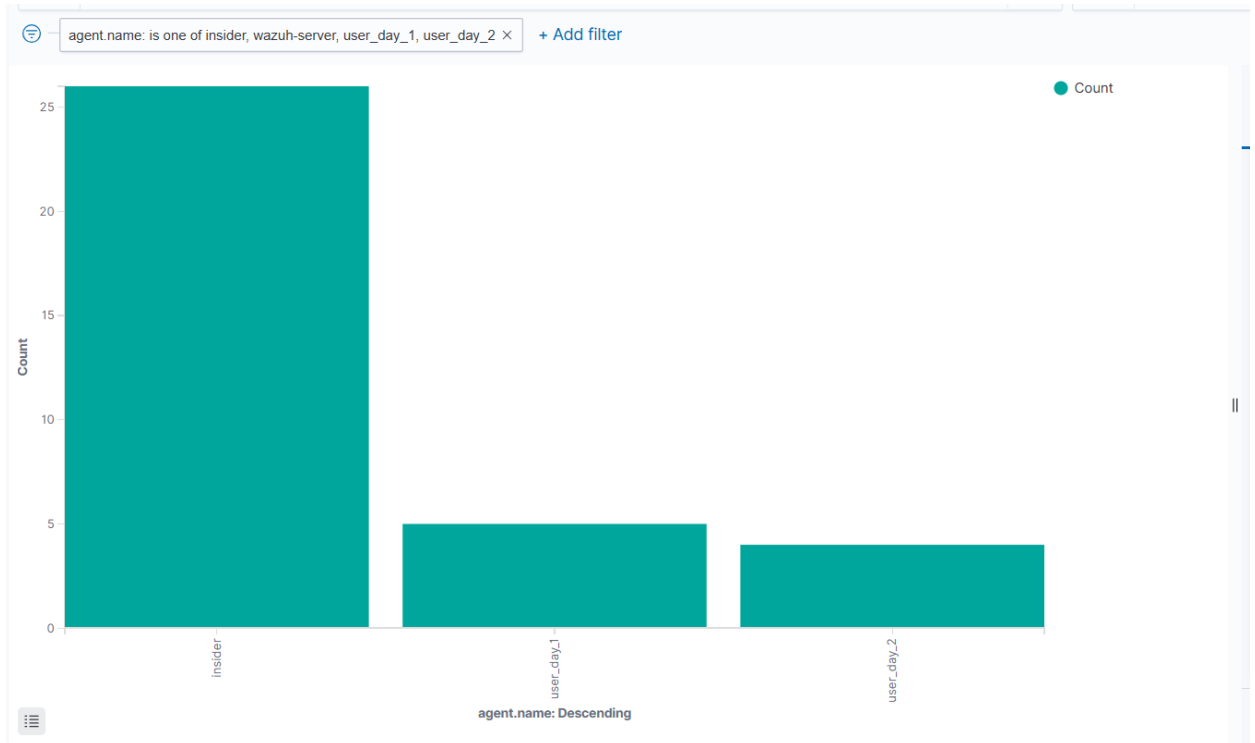


- Tables listing the detailed file events behind each alert for closer inspection.

The screenshot shows the Wazuh dashboard interface with the table view selected. The table lists detailed file events for the 'insider' agent. The columns are 'agent.name', 'syscheck.event', 'syscheck.path', and 'Total Syscheck Events'. The table is sorted by 'agent.name' in descending order. The sidebar on the right shows configuration options for 'wazuh-alerts-*', including aggregation, field, order by, and order settings.

agent.name: Descending	syscheck.event: Descending	syscheck.path: Descending	Total Syscheck Events
insider	modified	/data/insider_dump.bin	22
insider	modified	/data/insider_notes.txt	2
insider	added	/data/insider_dump.bin	1
insider	added	/data/insider_notes.txt	1
user_day_1	added	/data/new_file.txt	1
user_day_1	added	/data/u1_file.txt	1
user_day_1	modified	/data/u1_file.txt	2
user_day_1	deleted	/data/new_file.txt	1
user_day_2	added	/data/new_file.txt	1
user_day_2	added	/data/u2_file.txt	1

- A bar graph of agent name against event count, making the insider's spike easy to spot next to the quieter baseline users.



9. Business Use Cases

The same approach maps onto real organisational needs:

- Detecting employees who collect sensitive files before resigning or moving to a competitor.
- Giving a SOC team early warning of data staging before exfiltration happens.
- Supporting compliance and audit requirements by keeping a record of who touched sensitive data and when.
- Reducing reliance on manual log review, which is slow and easy to get wrong at scale.

10. Key Takeaways

A few lessons came out of building this lab:

- Insider threats need behaviour-based detection. Signatures alone miss them because the access is legitimate.
- File integrity monitoring is a strong sensor for this problem, since data theft almost always leaves a file trail.
- Unsupervised learning fits security data, where labelled examples of bad behaviour are scarce.

- Combining Wazuh for collection with a lightweight ML model for scoring gives a workflow that an analyst can actually use.

11. Future Improvements

There is room to take the lab further:

- Add network monitoring so file staging can be correlated with attempts to send data out.
- Expand the feature set with timing data, such as activity outside normal working hours.
- Tune the model against a larger and more varied baseline to cut false positives.
- Automate response, for example isolating an agent or alerting an analyst the moment an anomaly scores high enough.

12. Conclusion

This project set out to solve a specific problem: catching a trusted user who starts behaving in an untrusted way. Signature based tools miss that case because the access is legitimate and no rule gets broken. The lab answers it by watching file activity, learning each user's normal pattern, and flagging the events that do not fit.

The combination worked. Wazuh and Syscheck handled the sensor and collection side, capturing every file creation, modification, and deletion across the simulated users. The Isolation Forest model scored those events and isolated the insider's large file writes as anomalies, without anyone writing a rule to describe that exact behaviour. The Wazuh dashboard then made the result readable, so the insider's spike in activity stood out next to the quieter baseline users.

The wider lesson is that insider threats need behaviour-based detection, not signatures. File integrity monitoring turns out to be a strong sensor for it, since data theft almost always leaves a file trail, and unsupervised learning fits the reality that labelled examples of bad behaviour are rare. Pairing the two gives a workflow an analyst can actually use.

The lab is a starting point rather than a finished product. Adding network monitoring, richer features such as time of day, and automated response would move it closer to something deployable. As it stands, it shows that a lightweight, reproducible SOC environment can detect insider behaviour that traditional rules would let through.

13. References

- Wazuh Inc. (2026) *Wazuh documentation: File Integrity Monitoring (FIM)*. Available at: <https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/index.html> (Accessed: 26 June 2026).
- Wazuh Inc. (2026) *Wazuh documentation: Getting started with Wazuh*. Available at: <https://documentation.wazuh.com/current/getting-started/index.html> (Accessed: 26 June 2026).
- Liu, F.T., Ting, K.M. and Zhou, Z.-H. (2008) 'Isolation Forest', *2008 Eighth IEEE International Conference on Data Mining*. Pisa, Italy: IEEE, pp. 413–422.
- Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Docker Inc. (2026) *Docker documentation*. Available at: <https://docs.docker.com/> (Accessed: 26 June 2026).